

Moderation einbinden

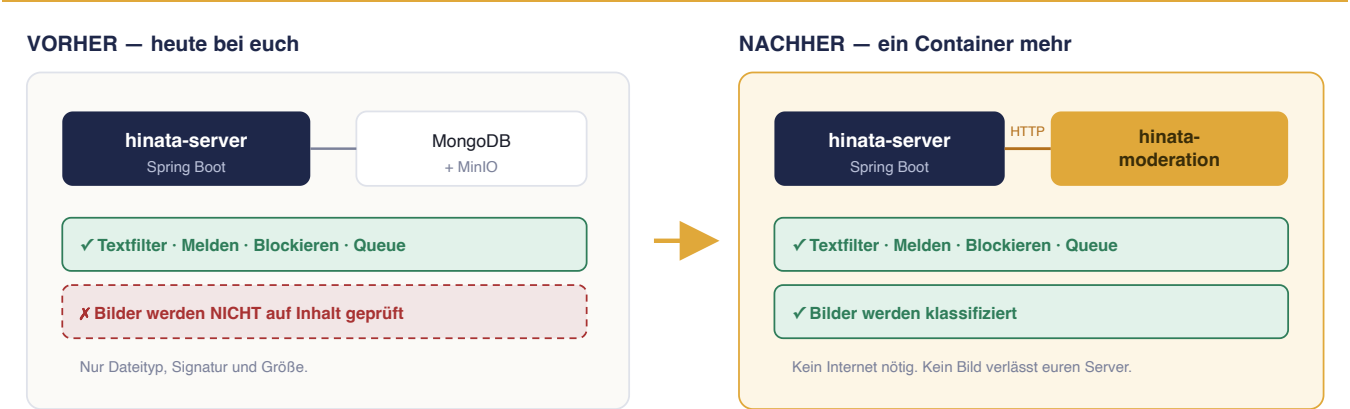
Wie `hinata-moderation` in eine laufende Hinata-Installation kommt — und wie man prüft, dass sie wirklich etwas tut.

Für alle, die eine eigene Hinata-Instanz betreiben · Stand 6. August 2026 · gilt ab der Server-Version, die das Moderations-Paket enthält

In einem Satz

Hinata prüft Texte ab Werk selbst; **Bilder prüft es erst, wenn ihr einen zweiten Container dazustellt**. Diese Anleitung stellt ihn dazu, verbindet ihn und zeigt, woran ihr erkennt, dass er tatsächlich klassifiziert und nicht nur läuft.

Was ihr schon habt — und was fehlt



Der Sidecar läuft im selben Compose-Netz. Er hat bewusst keinen veröffentlichten Port.

Der Klassifikator steckt *nicht* im Server-Image, weil ein ViT-Modell plus Laufzeit einige hundert Megabyte wiegt — die würde sonst auch jede Installation mitschleppen, die nie ein Bild bekommt.

Was ab Werk läuft

Funktion	Standard	Was sie tut
Textfilter	an	Deutsch + Englisch, über Titel, Beschreibungen, Kommentare, Artikel, Projekt- und Teamnamen. Kein Modell, kein Netz.
Melden & Blockieren	an	Jede Person kann Inhalte <i>und</i> Nutzer melden und blockieren.
Moderations-Queue	an	Adminbereich → Moderation.
Bild-Klassifikation (NSFW)	aus	Braucht den Sidecar. Das ist diese Anleitung.

Hash-Abgleich (bekanntes Material)	aus	Braucht Zugang bei einem der Programme — siehe Schritt 5.
Sprachnachrichten	ungeprüft	Werden gespeichert und ausgeliefert, ohne angesehen zu werden.
PDF / Office-Dateien	ungeprüft	Typ und Dateisignatur werden geprüft, der Inhalt nicht gelesen.

Zwei Dinge, die man leicht falsch liest

Der Textfilter ist absichtlich zurückhaltend. Lange Texte werden einem Menschen vorgelegt statt abgelehnt — ein zu Unrecht abgelehnter Bugreport wird nicht neu geschrieben, sondern aufgegeben. Kurze Felder, die alle sehen (Projektname), werden hart abgelehnt.

Zwei Kategorien kann keine Einstellung abschwächen: sexuelle Gewalt gegen Kinder und Schadsoftware. Dafür gibt es bewusst keinen Regler.

1 Sidecar dazustellen

Legt die Datei neben eure `docker-compose.yml` (oder mergt sie in ein vorhandenes Override). Eine fertige Fassung liegt im Repo unter `hinata-moderation/docker-compose.override.yml`.

```
# docker-compose.override.yml
services:
  hinata-moderation:
    image: ghcr.io/hinata-platform/hinata-moderation:1.0.0
    restart: unless-stopped
    networks: [hinata-moderation]

    # Kein `ports:` – bewusst. Von außen hat niemand hier etwas zu posten,
    # und ein veröffentlichter Port ist der Weg, auf dem ein Klassifikator
    # ohne Auth auf einer NAS im Internet landet, die auch 80/443 forwardet.
    security_opt: [ "no-new-privileges:true" ]
    cap_drop: [ ALL ]
    read_only: true
    tmpfs: [ "/tmp:size=64m,mode=1777" ]

    environment:
      MAX_BYTES: "16777216"          # 16 MB
      MAX_CONCURRENT: "1"
      MAX_QUEUED: "8"
      # INTRA_OP_THREADS: "2"        # auf einer NAS setzen: ONNX nimmt sonst alle Kerne

    deploy:
      resources:
        limits: { memory: 1g }
        reservations: { memory: 512m }

  hinata-server:
    networks: [hinata-moderation]

networks:
  hinata-moderation:
```

Vor dem Neustart einmal prüfen

Das Fragment *ergänzt* ein Netz beim `hinata-server`. Ob euer Basis-File dabei seine bestehenden Netze behält, sagt euch:

```
docker compose config | grep -A6 "hinata-server:"
```

Ein Server, der bei einem Merge sein Datenbank-Netz verliert, ist ein deutlich schlechterer Nachmittag als ein Klassifikator, der nicht verkabelt wurde.

```
docker compose up -d hinata-moderation
docker compose ps hinata-moderation    # muss "healthy" zeigen
```

2 Server verbinden — im Adminbereich

Ihr braucht dafür **keine** Umgebungsvariable und keinen Server-Neustart mehr. Der Endpunkt steht im Adminbereich und greift sofort.

Adminbereich › Moderation

Inhaltsprüfung
Moderation aktiv ☒
Bilder klassifizieren ☒

⚠ Kein Bild-Klassifikator installiert
Hochgeladene Bilder werden derzeit nicht geprüft.

Bild-Klassifikator
Adresse des Moderations-Sidecars
 Timeout

Schema des Panels (aus dem Widget-Code abgeleitet, kein Screenshot). Der Warnhinweis sitzt direkt über dem Feld, das ihn behebt.

Tragt als Adresse den Servicenamen aus eurem Compose-File ein, nicht `localhost` :

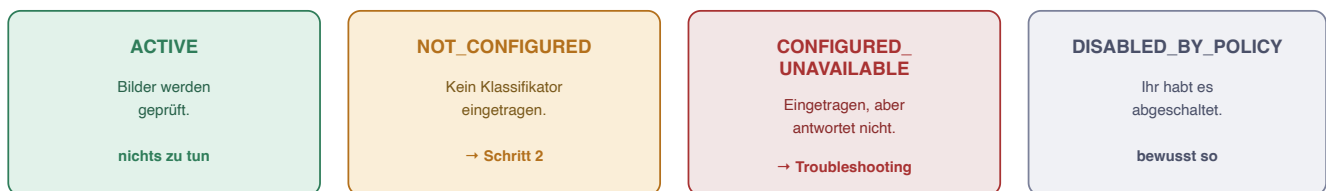
```
http://hinata-moderation:8081
```

Alternativ weiterhin per Umgebungsvariable möglich (`HINATA_MODERATION_IMAGE_ENDPOINT`). Was im Adminbereich steht, gewinnt — genau wie bei allen anderen Moderationseinstellungen.

3 Prüfen, dass wirklich klassifiziert wird

Der wichtigste Schritt

„Der Container läuft“ und „Bilder werden geprüft“ sind zwei verschiedene Tatsachen. Genau diese Verwechslung war der Grund, warum es im Adminbereich jetzt eine eigene Statuszeile gibt.



Die vier Zustände. Nur **ACTIVE** heißt, dass Bilder tatsächlich angesehen werden.

Drei unabhängige Wege, das zu bestätigen:

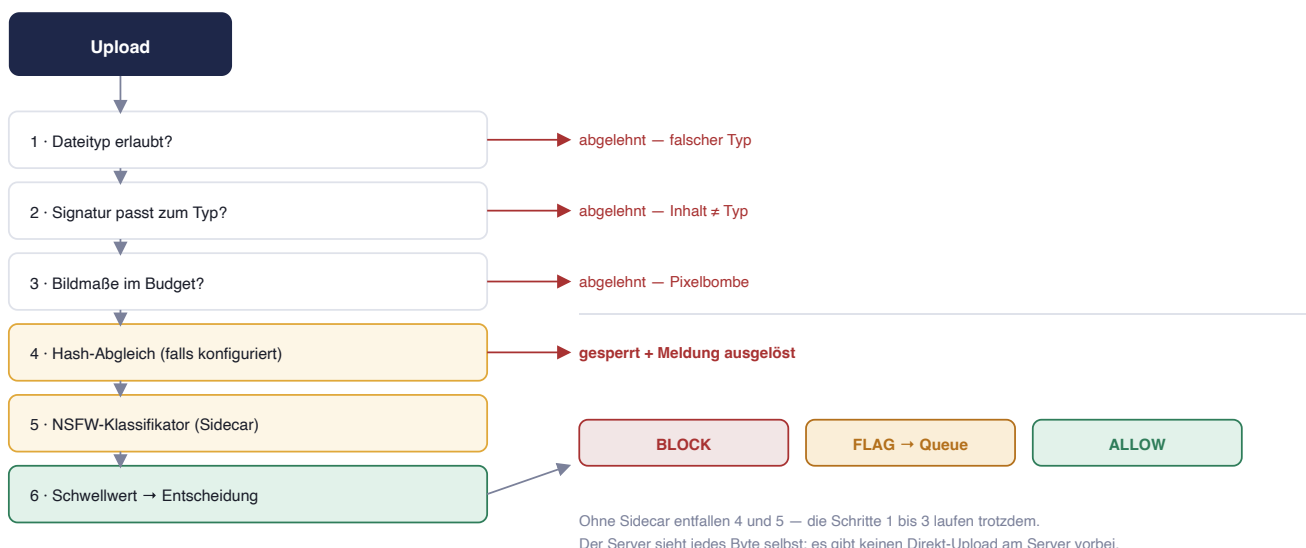
1. **Adminbereich → Moderation.** Die Zeile unter dem Schalter „Bilder klassifizieren“ nennt einen der vier Zustände.
2. **Server-Log beim Start.** Ist Bildmoderation an und kein Klassifikator erreichbar, steht dort eine **WARN**-Zeile, die den fehlenden Schlüssel benennt.
3. **Direkt am Sidecar** (aus einem Container im selben Netz):

```
docker compose exec hinata-server \
  curl -s http://hinata-moderation:8081/v1/health
```

Entscheidet euch bewusst

Wollt ihr *keine* Bildprüfung, schaltet den Regler im Adminbereich ab. Dann sagt das Panel „abgeschaltet“ statt eine Prüfung zu suggerieren, die nicht stattfindet — und niemand verlässt sich später darauf.

Was mit einem hochgeladenen Bild passiert



Schritt 3 schützt auch den Sidecar: eine Pixelbombe wird abgewiesen, bevor irgendetwas sie dekodiert.

4 Eskalations-Webhook einrichten

Damit erfahrt ihr *sofort*, dass etwas Schwerwiegendes gemeldet wurde — und nicht erst, wenn das nächste Mal jemand den Adminbereich öffnet. Adminbereich → Moderation → *Eskalation*:

Feld	Bedeutung
URL	Wohin die Benachrichtigung geht — Alerting, ein Chat-Webhook, ein Ticket-Eingang.
Secret	Signiert die Nachricht (X-Hinata-Signature , HMAC-SHA256). Pflicht — eine URL ohne Secret wird beim Speichern abgelehnt, und der Adapter verschickt nichts Unsigniertes. Eine Meldung „Inhalt eingefroren“, die der Empfänger niemandem zuordnen kann, ist schlechter als gar keine. Wird nie zurückgeliefert; ein leeres Feld heißt „gespeichertes behalten“, nicht „löschen“.
Timeout / Versuche	Zustellung mit begrenztem Retry. Fehlschläge landen im Audit-Log, werden nicht verschluckt.

Der Payload ist absichtlich langweilig

Er enthält eine Record-ID, eine Kategorie, die Oberfläche und einen Zeitstempel. **Keinen Inhalt, keinen Dateinamen, keine Bytes.** Deshalb dürft ihr ihn bedenkenlos in einen Chat-Kanal legen — die Meldung reist weiter, als der Inhalt je sollte.

5 Hash-Abgleich beantragen (optional, dauert Wochen)

Hinata hat einen Steckplatz dafür und liefert **keine Implementierung mit — und wird nie eine mitliefern.** Diese Programme prüfen die *betreibende Organisation*, nicht die Software; ihre Bedingungen verbieten die Weitergabe von Zugangsdaten; und ihre Hash-Listen liegen bewusst auf deren Servern, damit niemand eine lokale Kopie hat, gegen die sich Umgehung testen lässt.

Lohnt sich vor allem, wenn eure Instanz Inhalte von außerhalb der Organisation annimmt — etwa eine E-Mail-Adresse, die Tickets erzeugt.

- Microsoft **PhotoDNA Cloud Service** — microsoft.com/photodna
- **Thorn Safer** — safer.io
- Google **Child Safety Toolkit** — protectingchildren.google
- Cloudflare **CSAM Scanning Tool** — prüft nur, was durch Cloudflares Cache läuft; private Anhänge aus eurem eigenen Objektspeicher sieht es womöglich gar nicht. Vorher klären, bevor ihr euch darauf verlasst.

Die Zugangsdaten gehören danach euch und sind nicht übertragbar. Die Anbindung selbst ist ein kleiner Adapter gegen den vorhandenen Port.

Wenn die Eskalation auslöst

Schaut euch den Inhalt nicht an.

Das ist der wichtigste Satz dieser Anleitung. Hinsehen ist keine Überprüfung — es macht die Meldung nicht genauer, und bei einer Kategorie ist es selbst der Schaden. Die Software zeigt es euch auch nicht: eingefrorene Inhalte sind auch für Administratoren nicht erreichbar.



Einfrieren verbirgt, es löscht nicht. Beweise bleiben erhalten; der Zugriff endet — auch für Admins.

Hinata meldet für euch an niemanden. Es kennt eure Zuständigkeit nicht und ist nicht die meldende Stelle.

Vor dem Livegang: schreibt diese Kette auf und *benennt die Person*. Eine benannte Person mit Vertretung. „Die Admins“ ist um 23 Uhr am Freitag keine Antwort.

Troubleshooting

Symptom	Ursache und Behebung
NOT_CONFIGURED , obwohl der Container läuft	Der Endpunkt ist leer oder falsch. Servicename statt localhost verwenden. Prüfen: <code>docker compose exec hinata-server curl -s http://hinata-moderation:8081/v1/health</code>
CONFIGURED_UNAVAILABLE	Adresse erreichbar, aber der Sidecar meldet sich krank — meist ein fehlgeschlagener Modell-Ladevorgang. <code>docker compose logs hinata-moderation</code>
Uploads plötzlich langsam	Der Sidecar arbeitet mit einem Worker. Auf einer NAS <code>INTRA_OP_THREADS</code> setzen, damit ONNX nicht alle Kerne nimmt und mit der Datenbank konkurriert.
Bilder werden abgelehnt, die harmlos sind	Schwellwerte im Adminbereich. Der Regler für „lange Texte nur markieren“ betrifft nur Text, nicht Bilder.
Sidecar wird OOM-gekillt	Das Speicherlimit ist Absicht — dieser Container hält die angreifergesteuerten Pixelpuffer. Lieber er als die Datenbank daneben.

Was auch danach nicht geprüft wird

Damit ihr euch nicht auf etwas verlasst, das nicht passiert:

Bereich	Stand
Sprachnachrichten	Werden nicht transkribiert und nicht geprüft.
PDF- und Office-Inhalte	Nur Typ und Signatur, kein Textinhalt.
Videos	Keine Einzelbild-Analyse.
Statement of Reasons beim Einfrieren	Wird nicht automatisch erzeugt. Wenn ihr es braucht, ist es ein manueller Schritt in eurem Verfahren.
Bereits versandte Benachrichtigungen	Ein Einfrieren kann nicht zurückholen, was schon per Mail oder Push draußen ist. Es stoppt nur, was danach käme.

Warum diese Lücken bestehen

Keine davon ist ein Versehen. Jede ist eine Entscheidung, und die Gründe sind unterschiedlicher Natur: einer ist technisch, einer rechtlich, einer ist eine Grenze, die bewusst nie überschritten wird.

Sprachnachrichten — technisch machbar, rechtlich offen

Der Bau wäre überschaubar: ein zweiter Endpunkt am selben Sidecar, Whisper transkribiert, und der vorhandene Textfilter bewertet das Transkript. Kein neuer Bewertungspfad, keine neuen Kategorien.

Der Grund für die Zurückhaltung ist nicht der Aufwand, sondern die Rechtsgrundlage. Für *gehostete Dateien* gilt eine andere als für *zwischenmenschliche Kommunikation* — und eine Sprachnachricht an einem Vorgang ist Ersteres, eine in einer Direktnachricht womöglich Letzteres. Die einschlägige EU-Übergangsregelung (VO 2021/1232) lief am 3. April 2026 aus und wurde im Juli 2026 befristet bis 3. April 2028 neu erlassen, enger gefasst und mit menschlicher Prüfung, bevor etwas an eine Behörde geht. **Stand August 2026 — die dauerhafte Nachfolgeregelung ist noch nicht beschlossen, bitte selbst nachprüfen.** Das ist eine Frage für eure Rechtsberatung, keine für den Code.

Solange Hinata Sprachnachrichten nur als Anhang an Vorgängen kennt, stellt sich die Frage vermutlich nicht. Sobald es Sprach-Direktnachrichten gäbe, schon.

PDF und Office — der Parser ist das Risiko

Text und eingebettete Bilder aus einem PDF zu ziehen heißt, einen Parser auf eine Datei loszulassen, die jemand anderes gebaut hat. Alle drei bekannten Angriffe töten dabei den Serverprozess:

- komprimierte Container expandieren katastrophal — deshalb hat Apache Tika überhaupt einen `SecureContentHandler` ;
- die Bildextraktion aus PDFs läuft bei präparierten Masken in den Speicher (dokumentiert als PDFBOX-2101);
- der Bilddecoder alloziert nach den Maßen aus dem Header, *bevor* er sie prüft — genau die Lücke, die Schritt 3 oben schließt.

Machbar ist das also nur außerhalb des Serverprozesses, mit harten Grenzen für Speicher, Laufzeit und Ausgabegröße: ein weiterer Container, kein Konfigurationsschalter. Dazu kommt, dass Tikas Bomben-Heuristik dokumentierte Fehlalarme hat (TIKA-2091) — ein Treffer dürfte also nicht ablehnen, sondern nur einem Menschen vorlegen.

Video — die Kosten müssen begrenzt sein, bevor man anfängt

Der übliche Weg ist Einzelbild-Abtastung bei rund einem Bild pro Sekunde. Ohne harte Obergrenze wird ein zweistündiger Upload still zu 7.200 Klassifikationen. Es braucht also ffmpeg in einem eigenen Container, eine feste Obergrenze an Bildern und eine Entdopplung ähnlicher Bilder — und die Ehrlichkeit, dass ein Bild pro Sekunde ein für 300 ms eingeblendetes Bild nicht sieht.

Kein Statement of Reasons beim Einfrieren — als Lücke markiert, nicht kaschiert

Das Datenmodell führt ein Feld dafür, das nie gesetzt wird, und der Code sagt genau das. Die Alternative wäre ein Feld, das `true` behauptet und damit einen Schritt vortäuscht, den niemand ausführt — schlechter als eine offen benannte Lücke. Wenn diese Pflicht für euch greift, ist sie ein Punkt in eurem Verfahren, nicht in der Software.

Kein eigener CSAM-Klassifikator — und das ändert sich nie

Ein solches Modell zu trainieren setzt voraus, das Material zu besitzen. Das ist die Straftat selbst. Ein allgemeines NSFW-Modell dafür zweckzuentfremden ist schlimmer als nichts: es schlägt bei gewöhnlichen Familienfotos an, und jeder Fehlalarm erzeugt eine menschliche Sichtung genau des Materials, dessen Sichtung das System verhindern soll.

Deshalb der Steckplatz für die geprüften Programme aus Schritt 5 statt eines eigenen Modells — und deshalb liegen deren Hash-Listen auf deren Servern und nicht in diesem Image: eine lokale Kopie wäre ein Orakel, gegen das sich Umgehung testen lässt.

Bereits versandte Benachrichtigungen

Eine Mail, die zugestellt ist, ist zugestellt. Ein Einfrieren stoppt, was danach käme, und markiert den gespeicherten Verlauf — zurückholen kann es nichts. Das ist keine Designentscheidung, sondern eine Eigenschaft von E-Mail.

Und die rechtliche Seite?

Steht bewusst nicht hier, sondern als *Fragen* in `OPERATORS.md` im Server-Repo — weil die Antworten davon abhängen, wie eure Instanz konfiguriert ist, und weil sich das Feld schnell bewegt. Ein Dokument, das sie als feststehend ausgibt, ist für einen Teil der Betreiber sofort falsch und für alle irgendwann.

Ein Satz gilt trotzdem immer: wer hostet, trägt die Pflichten. Das seid ihr, nicht das Hinata-Projekt.